

AI ADOPTION • RESPONSIBLE AUTOMATION • DESKTOP IMPLEMENTATION

# AI Job Hunt System

*A dual-interface job-search system: specialised Claude Code CLI agents share a Supabase/MCP pipeline with a newer human-gated macOS desktop application.*

## CURRENT STATUS

**Working personal system; shared MCP and desktop validated, automation repairs required**

## EVIDENCE REVIEWED

Evidence basis: the adapted Claude CLI toolkit and separate desktop-app repositories, their source, configuration, tests and contribution history, plus validation rerun on 26 August 2026. The toolkit snapshot includes uncommitted backend changes.

## Problem

A serious job search can become fragmented across role discovery, database records, local career documents, browser forms and remembered workflows. A command-line system can orchestrate deep specialist work, but pipeline state and readiness are harder to see; AI-generated changes also create risks around unsupported claims, stale documents and consequential actions taken without review.

## Objective

Retain the Claude-based CLI as the specialist execution layer while adding a calm desktop view over the same Supabase/MCP domain. Both interfaces should preserve one source of truth, attribution and explicit gates; the GUI should improve daily adoption without weakening the CLI's coaching, discovery, document and outreach workflows.

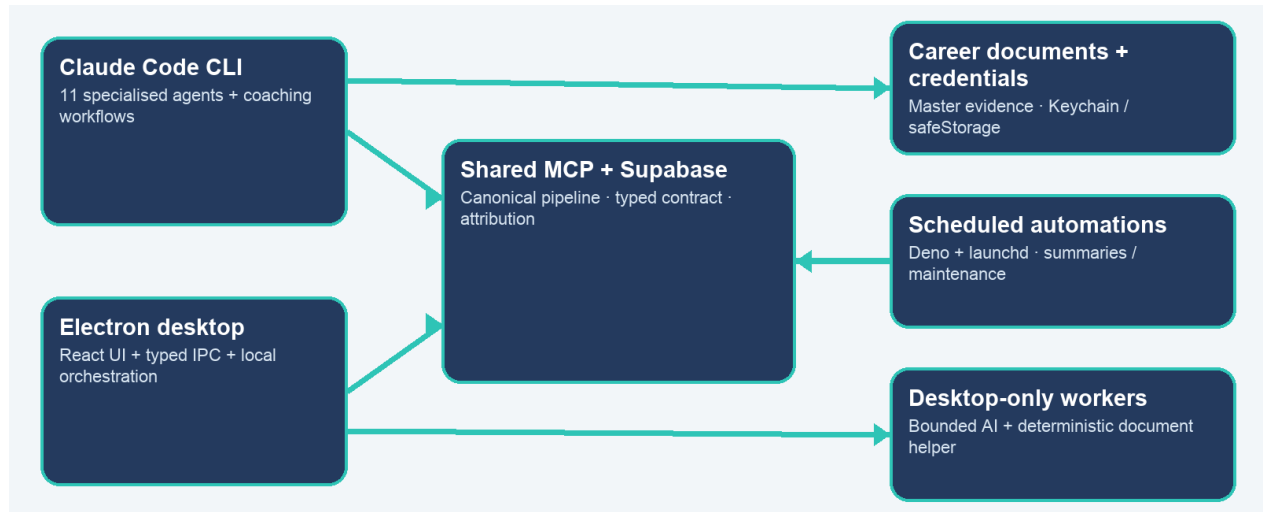
## WHY THIS MATTERS

This is a practical adoption story rather than a single-interface app. It combines an agentic CLI for specialist workflows, a shared data contract and a desktop interface for visibility and safer routine use - while keeping human control at application and document gates.

## SYSTEM VIEW

## Architecture and workflow

The original operating model is Claude Code plus specialised agent definitions, coaching frameworks and MCP tools. The desktop app is a second client: its unprivileged renderer uses typed IPC, while the Electron main process coordinates the same MCP boundary plus local AI and document workers.



Architecture reconstructed from the reviewed implementation artefacts.

## Operational workflow

1. Start in the Claude job-coach or desktop overview and read the shared canonical pipeline state.
2. Discover or add roles, de-duplicate them and triage priority/rank before creating an application.
3. Use CLI specialists for research, resumes, cover letters, outreach and application guidance, or run the desktop's bounded resume/review workflows.
4. Ground document changes in master career evidence and pause for approval before applying edits.
5. Run an independent pre-submission review; the desktop variant ties the pass to exact file hashes.
6. Check networking and review gates, with any override explicitly reasoned and logged.
7. Hand final submission to the user: the desktop opens an HTTPS site for manual completion; the CLI contains Playwright form-filling instructions but still requires a final confirmation gate.

## FOUNDATIONS

## Technologies and controls

*The stack below reflects the reviewed implementation; legacy or optional paths are identified explicitly.*

| AREA            | EVIDENCED TECHNOLOGY / PATTERN                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------------|
| CLI and agents  | Claude Code, 11 specialised agent definitions, coaching playbooks and project templates                  |
| Shared platform | Supabase PostgreSQL, Edge Function MCP server, Zod contracts, attribution and idempotency                |
| Automation      | Deno, macOS launchd, Slack/email notifications, optional Playwright enrichment                           |
| Desktop         | Electron 43, React 19, TypeScript 7, Vite 8, Electron Forge                                              |
| Desktop AI      | Provider-neutral worker with fixed OpenAI, Anthropic and Google adapters                                 |
| Documents       | Master Markdown/DOCX evidence, bundled Python helper, python-docx, pypdf, LibreOffice                    |
| Validation      | Deno tests/checks, Vitest, Playwright, fixture MCP server, TypeScript compiler                           |
| Security        | Header-only MCP authentication, Keychain/safeStorage, CSP, sandboxing, path containment, encrypted cache |

**DELIVERY**

# What was actually built

*Evidence labels separate implemented work from claims, plans and dependencies.*

**ADAPTED FOUNDATION Claude Code job-hunt toolkit**

The repository is a fork of Daniel Frysinger's toolkit. It provides specialised agents, coaching frameworks, templates, a Supabase MCP server and scheduled scripts; this is not presented as wholly original work.

**CONTRIBUTOR IMPLEMENTATION UK discovery and credential additions**

Contribution history and README support the Adzuna agent, root discovery configuration, job-vault Keychain helper and public-distribution security work.

**CONTRIBUTOR IMPLEMENTATION Independent application reviewer and gate**

A read-only reviewer cross-checks the exact application package; a current pass and networking start are required before application unless a reasoned override is logged.

**CONTRIBUTOR IMPLEMENTATION Desktop-compatible MCP contract**

Shared handlers add structured envelopes, typed errors, optimistic concurrency, atomic ranking, idempotency, exact document hashes and append-only run events while retaining legacy CLI tools.

**IMPLEMENTED Electron desktop client**

Overview, jobs, pipeline, applications, interviews, contacts, activity and settings are wired through a typed preload API to the shared MCP backend.

**IMPLEMENTED Secure, bounded desktop AI and documents**

Provider-neutral resume/review workers expose no tools; safeStorage, approved workspace paths and a deterministic document helper constrain secrets and file changes.

**IMPLEMENTED WITH CAVEAT CLI application and automation workflows**

Agent instructions cover discovery, tailoring, outreach and Playwright-assisted forms; launchd scripts cover summaries and maintenance, but live connectors were not exercised and the automation layer is not currently type-clean.

**PLANNED Scheduled daily job discovery**

The repository contains a configuration format, but README and changelog explicitly state that the daily-job-discovery runner is not implemented.

## Implementation notes

- The CLI uses separate agent definitions so coaching, discovery, resume, cover letter, review, outreach and application work have narrower instructions.
- Both clients use the same Supabase records and MCP rules; the desktop contract extends the server without renaming legacy fields or removing legacy tool responses.
- The desktop renderer receives asset identifiers and workspace-relative paths rather than arbitrary filesystem access, and its AI worker exposes no shell, file or web tools.
- Resume changes are grounded against supplied evidence; desktop changes require exact before/after approval and review freshness is tied to cryptographic hashes.
- CLI ATS credentials use macOS Keychain through job-vault; desktop MCP/provider secrets use Electron safeStorage.
- The optional LinkedIn enrichment and maintenance scripts are disabled by default and the repository explicitly warns that they can breach LinkedIn's terms.

## ASSURANCE

## Testing and validation evidence

*Checks were rerun where the local repository and dependencies made that practical.*

| CHECK                           | RESULT         | EVIDENCE                                                                                                                                                |
|---------------------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Shared MCP type-check           | PASS           | The local MCP endpoint and deployed Supabase Edge Function both passed Deno checks on 26 August 2026.                                                   |
| Shared MCP tests                | PASS           | Six Deno tests passed: transitions, optimistic concurrency, review freshness, exact package selection, correlation and legacy envelopes.                |
| Desktop type safety             | PASS           | The separate desktop repository completed its TypeScript compiler check with no errors.                                                                 |
| Desktop unit suite              | PASS           | 32 tests passed; two gated tests were skipped in the default run.                                                                                       |
| Desktop MCP contract            | PASS           | Two integration tests passed against a local Streamable HTTP fixture.                                                                                   |
| Desktop end-to-end              | PASS           | Three Playwright tests passed, including onboarding isolation and a complete audited manual-application flow.                                           |
| Desktop performance budgets     | PASS (fixture) | The fixture met cached visibility (<1s) and live refresh (<2s) thresholds; these are not production benchmarks.                                         |
| Scheduled automation type-check | FAIL           | The Deno automation check reports 24 current type errors, mainly Supabase joined-result casts and missing DOM library types in browser evaluation code. |
| Live agent/connectors           | NOT RETESTED   | No end-to-end live Indeed, LinkedIn, Adzuna, notification or ATS application run was performed during this review.                                      |

### INTERPRETATION

A passing automated check supports the stated implementation behaviour; it does not prove external adoption, live production performance or business impact unless that evidence is separately present.

## REFLECTION

# Limits, lessons and next steps

## Limitations

- The toolkit is an adapted open-source fork; portfolio claims must distinguish upstream foundations from later contributions.
- The scheduled automation layer currently fails Deno type-checking with 24 errors and has no automated test suite in the repository.
- Live job-board APIs, notifications and Playwright form flows were not exercised; scheduled daily discovery is documented intent rather than an implemented runner.
- Optional LinkedIn scraping/enrichment is explicitly warned as terms-of-service risk and disabled by default.
- The desktop is a personal macOS MVP requiring the shared backend at contract 1.3.0; it is not yet a signed, notarised public release and fixture timing is not production evidence.
- The desktop does not fill or submit employer forms, create cover letters/outreach, or expose arbitrary shell/filesystem tools.

## Lessons learned

- A useful interface strategy can be additive: retain the CLI's specialist depth while using a GUI to improve visibility and adoption.
- Human approval is an architectural boundary, not a final UI checkbox.
- AI output becomes safer when structured planning/review is separated from deterministic file mutation and final submission.
- Shared contracts need backward compatibility, while prompt/configuration artefacts need live tests and clear fork attribution before broader claims are made.

## Next steps

1. Repair the 24 Deno type errors and add repeatable tests for scheduled summary, enrichment and maintenance scripts.
2. Add controlled fixture or sandbox tests for job-board discovery, notifications and Playwright-assisted application flows.
3. Apply/verify the forward-only backend migrations and deploy the matching MCP function before production desktop writes.
4. Complete a signed and notarised macOS release and run usability testing before claiming adoption improvements.
5. Decide whether daily discovery is still desirable; prefer official APIs, keep scraping opt-in, and add recovery tests around the shared audit trail.

## Evidence base reviewed

- Toolkit source/docs: README, changelog, agents, coach tools, templates, extension, MCP, migrations and six Deno tests
- Toolkit Git history/remotes: upstream dfryfinger fork plus 18 later contribution commits by Yogi
- Desktop source/docs: README, PRD, Electron app, document helper and unit/contract/end-to-end tests
- Toolkit commits c9fdbfc through b21ffc5; desktop commits 03d1083 through e32e5d5

**CLAIM DISCIPLINE** *Implemented claims are restricted to reviewed artefacts; planned or externally dependent items are labelled.*